

Role and Authorization Management

The StockWise application implements a sophisticated, centralized security model based on the principle of **Least Privilege**. We utilize a hierarchical **Role-Based Access Control (RBAC)** implementation within our Spring Boot monolithic architecture to ensure strict separation of duties and secure resource management across the application lifecycle.

3.1 Role-Based Access Control (RBAC) Architecture

Our RBAC implementation is integrated directly into the Spring Security configuration, providing a unified security layer for the entire application.

- **Hierarchical Roles:** The system defines discrete roles—specifically `ROLE_ADMIN` and `ROLE_STAFF`. Access permissions are mapped to these roles, ensuring that business operations are authorized based on the user's assigned scope.
- **Granular Policy Enforcement:** We employ **Policy-Based Authorization**. This allows us to define fine-grained access rules that restrict navigation and action availability within the UI, ensuring that users only interact with permitted system components.

3.2 Web-Level Access Control (Method Security)

To ensure maximum security coverage, we enforce authorization checks at the controller level:

- **Centralized Filter Chain:** We utilize the `SecurityFilterChain` bean to intercept all incoming HTTP requests. Any request lacking a valid session or the required clearance level is rejected at the perimeter with an **HTTP 403 (Forbidden)** response.
- **Method-Level Security:** Within our Spring MVC Controllers and Service layers, we utilize Spring Security's **SpEL (Spring Expression Language)** via `@PreAuthorize` and `@Secured` annotations. For example, sensitive administrative operations—such as deleting products or viewing global inventory analytics—are explicitly locked behind method-level guards that validate the `Principal`'s role before any database transaction is initiated.

3.3 Secure Session Management

We have adopted a **Stateful Security Architecture** (Session-based) to manage user sessions efficiently within our monolithic Spring Boot application.

- **Session Lifecycle:** Upon successful authentication, the server creates an `HttpSession` and issues a secure `JSESSIONID` cookie to the client's browser. This session maintains the authenticated state on the server, ensuring that the user remains logged in throughout their interaction with the system.
- **Security Context Propagation:** The `SecurityContext` is automatically retrieved from the `HttpSession` for every request. This allows Spring Security to identify the authenticated user and their assigned roles without requiring repeated credential verification.

- **Session Integrity & Security:** Since our architecture is stateful, the session is managed entirely server-side. We configure our session management to ensure that session identifiers are regenerated upon login (to prevent session fixation) and that cookies are marked as `HttpOnly` and `Secure`, protecting the application against common vulnerabilities.